TECHNICAL ARCHITECTURE DOCUMENT

Bulk SMS Platform Rebuild

Comprehensive technical blueprint for building a modern SMS platform reusing the existing sdasms_sdasms MySQL database with 65 tables, covering database analysis, API architecture, and migration strategy.

Database: sdasms_sdasms | Tables: 65 | Stack: Next.js + Node.js

APRIL 2026 VERSION 1.0

Table of Contents

1. Executive Summary 2

2. Existing Database Analysis 2

2.1 Detected Technology Stack . . . 3

2.2 Database Module Overview . . . 3

2.3 System Architecture Overview . . . 4

2.4 Database Module Relationships . . . 5

3. Core Table Analysis 6

3.1 User Management (cg_users) . . . 6

3.2 SMS Campaigns (cg_campaigns) . . . 7

3.3 Sending Servers (cg_sending_servers) . . . 7

3.4 Contacts and Groups . . . 8

3.5 Billing and Subscriptions . . . 8

4. Proposed Technology Stack 8

5. API Architecture 9

5.1 Core API Endpoints . . . 10

6. Frontend Module Design 11

7. Database Access Strategy 12

7.1 Recommended: API Middleware Layer . . . 12

8. Migration Strategy 12

8.1 Phase 1: Foundation (Weeks 1-3) . . . 13

8.2 Phase 2: Core Features (Weeks 4-8) . . . 13

8.3 Phase 3: Billing and Admin (Weeks 9-11) . . . 13

8.4 Phase 4: Testing and Launch (Weeks 12-14) . . . 13

9. Security Considerations 14

10. Development Roadmap 14

1. Executive Summary

This document presents the comprehensive technical architecture for rebuilding a modern Bulk SMS platform that reuses the existing sdasms_sdasms MySQL database. The existing platform is a production-grade, multi-tenant SMS SaaS application built with Laravel (PHP) and hosted on cPanel, comprising 65 database tables across 13 functional modules. The new platform will be built using Next.js for the frontend and Node.js for the backend API layer, connecting to the same MySQL database to preserve all existing data, user accounts, campaigns, contacts, billing records, and SMS delivery history.

The existing database supports a rich feature set including multi-channel messaging (SMS, Voice, MMS, WhatsApp, Viber, OTP), campaign scheduling with recurring automations, two-way chat conversations, India DLT compliance, role-based access control, subscription-based billing with tiered pricing, sender ID management, contact segmentation, and comprehensive delivery reporting. The new platform must maintain full compatibility with all 65 tables while delivering a significantly improved user experience through a modern, responsive web interface built with Next.js and React Server Components.

The recommended architecture follows a middleware API pattern, where the new Next.js application communicates with the MySQL database through a Node.js API layer. This approach provides better security by eliminating direct database exposure, enables caching and rate limiting, and allows for incremental migration without disrupting the existing cPanel-hosted platform. Both platforms can operate simultaneously against the same database during the transition period, ensuring zero downtime for existing users.

Metric	Value
Database Name	sdasms_sdasms
Total Tables	65
Existing Platform	Laravel/PHP on cPanel
New Frontend	Next.js 15 (React)
New Backend	Node.js + Express/Fastify
Database Engine	MySQL 8.0+
Messaging Channels	SMS, Voice, MMS, WhatsApp, Viber, OTP
Compliance	India DLT (TRAI)

Table 1. Project Overview Metrics

2. Existing Database Analysis

The existing database follows a consistent Laravel convention with all tables prefixed by cg_ (likely standing for "CommGateway" or the commercial product name). Every table includes an auto-incrementing bigint primary key, a char(36) UUID field named uid for external references, and automatic created_at/updated_at timestamp columns. This consistent pattern greatly simplifies the task of building a new application layer, as the data model is well-structured and predictable across all modules.

2.1 Detected Technology Stack

Several indicators in the database schema reveal the underlying technology choices of the existing platform. The presence of a cg_migrations table storing Laravel migration file names and batch numbers confirms the use of the Laravel PHP framework. The cg_jobs, cg_job_batches, and cg_failed_jobs tables are the standard Laravel Queue infrastructure for background job processing. The cg_personal_access_tokens table follows the exact schema of Laravel Sanctum, the official Laravel API authentication package, indicating that the existing platform already provides REST API access for external integrations.

Evidence	Technology	Implication
cg_migrations table	Laravel Framework	PHP backend with Eloquent ORM
cg_jobs, cg_job_batches	Laravel Queue	Background job processing for bulk SMS
cg_personal_access_tokens	Laravel Sanctum	Existing REST API with token auth
uid char(36) on every table	UUID pattern	Laravel Str::uuid() for public IDs
dlt_template_id, dlt_entity_id	India DLT	TRAI regulatory compliance built-in
Multi-channel fields	Omnichannel	SMS, Voice, MMS, WhatsApp, Viber, OTP

Table 2. Technology Stack Detection from Database Schema

2.2 Database Module Overview

The 65 tables are organized into 13 functional modules, each serving a distinct business capability. Understanding these modules is critical for the new platform development, as each module maps to a specific set of API endpoints, UI pages, and business logic components. The following table provides a complete inventory of all modules with their table counts and primary business functions.

Module	Tables	Key Tables
Users and Auth	5	cg_users, cg_roles, cg_role_user, cg_permissions, cg_password_resets

SMS Campaigns	5	cg_campaigns, cg_campaigns_lists, cg_campaigns_senderids, cg_automations, cg_file_campaign_data
Contacts Management	7	cg_contacts, cg_contact_groups, cg_contact_group_fields, cg_contacts_custom_field
Two-Way SMS/Chat	2	cg_chat_boxes, cg_chat_box_messages
Reporting and Tracking	3	cg_reports, cg_tracking_logs, cg_schedule_messages
Sending Servers	4	cg_sending_servers, cg_custom_sending_servers, cg_sending_server_based_pricing_plans
Sender IDs	2	cg_senderid, cg_senderid_plans
Billing and Subscriptions	9	cg_plans, cg_subscriptions, cg_invoices, cg_payment_methods, cg_customer_based_pricing_plans
Templates	3	cg_templates, cg_template_tags, cg_email_templates
Keywords and Numbers	2	cg_keywords, cg_phone_numbers
Segmentation	2	cg_segments, cg_segment_conditions
Job Queue System	5	cg_jobs, cg_job_batches, cg_job_monitors, cg_failed_jobs, cg_import_job_histories
System and Utilities	16	cg_app_config, cg_countries, cg_currencies, cg_languages, cg_spam_word, cg_blacklists, cg_plugins, cg_notifications

Table 3. Complete Database Module Inventory (65 Tables)

2.3 System Architecture Overview

The proposed system architecture follows a three-tier design pattern with a clear separation between the presentation layer, the application logic layer, and the data layer. The existing cPanel-hosted Laravel application and the new Next.js platform will share the same MySQL database during the transition period, with both writing to and reading from the identical set of 65 tables. A Redis instance handles job queuing and real-time WebSocket communication for the two-way chat feature, while multiple SMS gateway integrations connect through configurable sending server definitions.

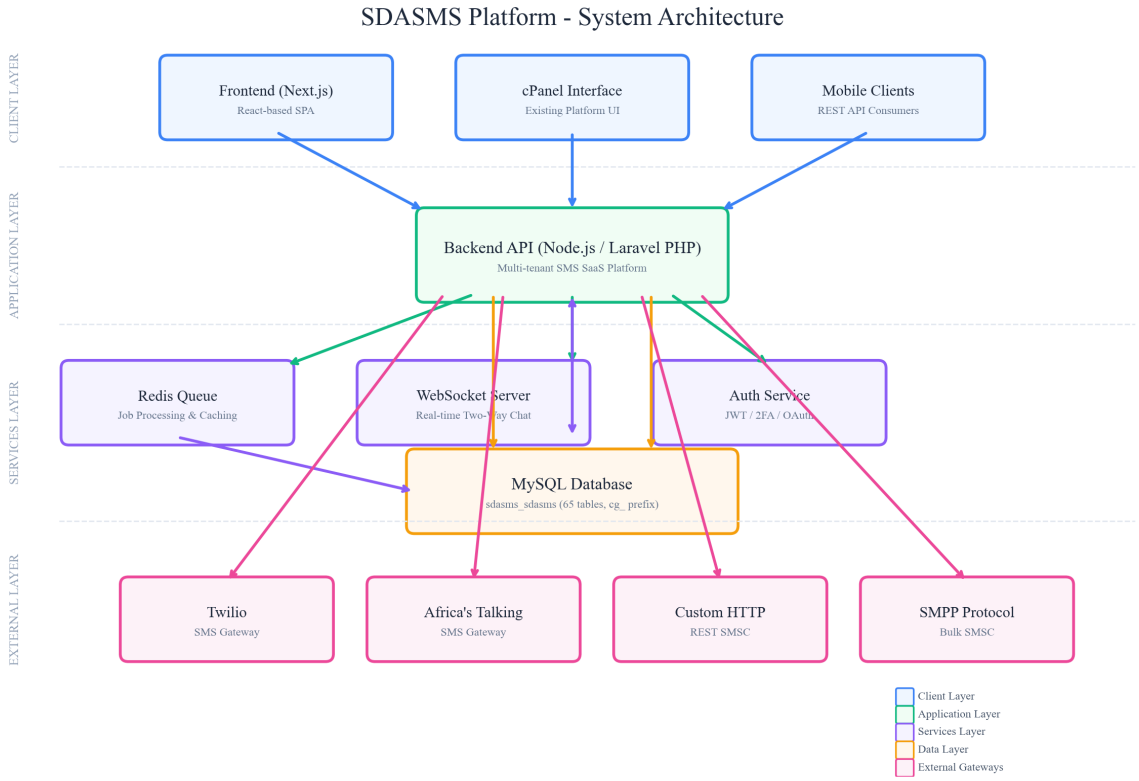

Figure 1. System Architecture Overview

2.4 Database Module Relationships

The 13 database modules are interconnected through foreign key relationships centered on the cg_users table, which serves as the central entity linking almost every module. Each user can own campaigns, contacts, chat conversations, sender IDs, phone numbers, and subscriptions. The campaign module connects to contacts through many-to-many relationships, and the tracking module records delivery outcomes for every message sent through any campaign or automation. The billing module manages subscription lifecycles and invoice generation, while the sending server module provides the gateway abstraction layer that routes messages through different providers based on country-specific pricing and availability.

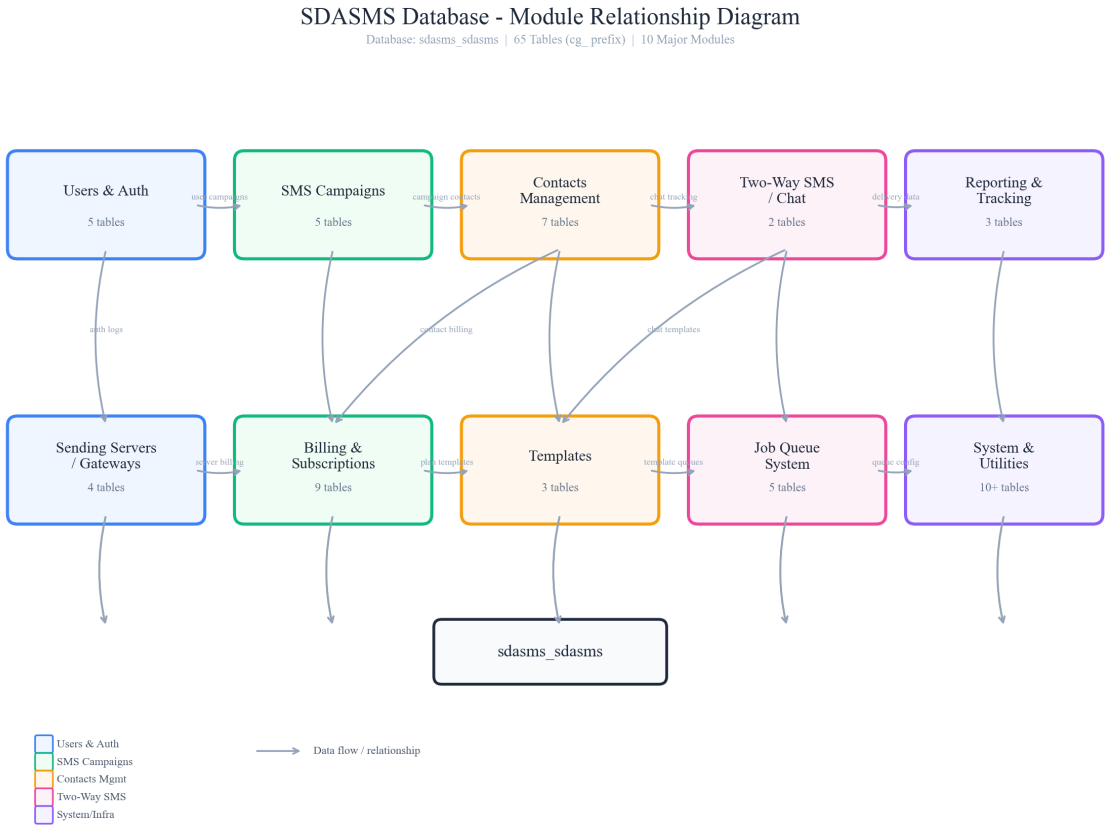

Figure 2. Database Module Relationship Diagram

3. Core Table Analysis

This section provides detailed analysis of the most critical tables in the database. These tables form the backbone of the SMS platform and must be fully understood before building the new application layer. Each subsection covers the table structure, key columns, business logic implications, and integration requirements for the new platform.

3.1 User Management (cg_users)

The cg_users table is the central entity of the entire platform. It supports a multi-tenant architecture through a parent_id column that enables hierarchical user relationships (admin creates sub-accounts, resellers create customer accounts, etc.). The table includes both is_admin and is_customer boolean flags to distinguish between system administrators and paying customers, while the cg_roles and cg_permissions tables provide a more granular role-based access control system. The api_token column stores a unique token for each user, enabling direct API access without requiring full OAuth flows. Two-factor authentication is supported through two_factor, two_factor_code, two_factor_expires_at, and two_factor_backup_code columns. DLT compliance fields (dlt_entity_id, dlt_telemarketer_id) indicate India-specific regulatory requirements, while the sms_unit column tracks the user's remaining

SMS credit balance.

Column	Type	Purpose
id, uid	bigint, char36	Primary key and public UUID
parent_id	bigint	Multi-tenant hierarchy (admin/reseller/customer)
email, password	varchar191	Authentication credentials
is_admin, is_customer	tinyint	Role flags for access control
api_token	varchar191	API authentication token
sms_unit	varchar191	Remaining SMS credit balance
two_factor	tinyint, int, datetime	Two-factor authentication support
dlt_entity_id	varchar191	India DLT regulatory compliance
webhook_url	varchar191	Delivery receipt webhook endpoint
api_sending_server	bigint	Default gateway for API requests

Table 4. cg_users Key Columns

3.2 SMS Campaigns (cg_campaigns)

The cg_campaigns table is the heart of the SMS platform, supporting both one-time and recurring campaign scheduling with sophisticated targeting capabilities. Each campaign can be associated with multiple contact lists through the cg_campaigns_lists junction table, and can use multiple sender IDs through cg_campaigns_senderids. The scheduling system supports onetime and recurring schedule types, with recurring campaigns configurable through frequency_cycle, frequency_amount, and frequency_unit columns. The running_pid column tracks the background process ID for active campaigns, enabling process management and monitoring. The dlt_template_id column links each campaign to an approved DLT template for regulatory compliance, and skip_failed_message controls whether failed recipients should be retried in subsequent batches.

3.3 Sending Servers (cg_sending_servers)

The cg_sending_servers table provides a comprehensive abstraction layer for SMS gateway connectivity, supporting multiple protocols including HTTP API, SMPP, WhatsApp, Viber, and OTP-specific gateways. Each sending server configuration stores credentials across numerous authentication schemes (API key/secret, OAuth tokens, SMPP username/password, AWS access keys, and more) to accommodate the diverse authentication mechanisms used by different SMS providers. The table includes rate limiting configuration through sms_per_request, quota_value, quota_base, and quota_unit columns, enabling precise control over message throughput per gateway. The custom flag

indicates whether this server uses the standard integration or a custom HTTP configuration defined in cg_custom_sending_servers, which maps request parameters to the specific API format required by each provider.

3.4 Contacts and Groups

The contacts module provides a sophisticated contact management system built around cg_contact_groups (also referred to as contact lists). Each contact group supports custom fields through cg_contact_group_fields, which defines field types, labels, tags, and validation rules. Individual contacts in cg_contacts are linked to both a customer (owner) and a group (membership), with custom field values stored in cg_contacts_custom_field. The opt-in/opt-out system uses dedicated keyword tables (cg_contact_groups_optin_keywords and cg_contact_groups_optout_keywords) to automatically manage group subscriptions via SMS keywords. Contact groups also support automated welcome messages, unsubscribe notifications, and signup confirmations, making them function as self-service subscription lists for end recipients.

3.5 Billing and Subscriptions

The billing module implements a complete subscription-based monetization system. The cg_plans table defines pricing plans with configurable billing cycles, while cg_plans_coverage_countries maps per-country SMS rates and gateway assignments to each plan. Tiered credit pricing through cg_plan_sending_credit_prices enables volume-based discounts (e.g., first 1000 credits at $0.01 each, next 5000 at $0.008). The cg_subscriptions table tracks active subscriptions with status workflow (new, pending, active, ended, renew), while cg_subscription_transactions records payment history. The cg_customer_based_pricing_plans table allows custom pricing overrides per customer, enabling enterprise or reseller-specific rate agreements. Invoices are managed through cg_invoices with support for tax calculation, multiple payment methods, and transaction tracking.

4. Proposed Technology Stack

The new platform will be built with a modern technology stack optimized for performance, developer productivity, and long-term maintainability. The choice of Next.js for the frontend and Node.js for the backend provides a unified TypeScript development experience across the entire application, reducing context switching and enabling code sharing between client and server. The following table details each technology choice and its justification in the context of building a bulk SMS platform that must integrate with an existing MySQL database.

Layer	Technology	Justification

Frontend	Next.js 15 (React)	SSR, file-based routing, React Server Components, excellent DX
UI Components	shadcn/ui + Tailwind CSS	Accessible, customizable, consistent design system
Backend API	Node.js + Fastify	Fast, schema-based validation, plugin ecosystem, TypeScript native
Database	MySQL 8.0 (existing)	Reuse existing sdasms_sdasms database without migration
ORM	Prisma or Drizzle	Type-safe database access, migration management
Queue/Jobs	Redis + BullMQ	Background SMS processing, retry logic, job monitoring
Real-time Chat	Socket.io / WebSocket	Two-way SMS chat with live message delivery
Authentication	NextAuth.js / JWT	Compatible with existing cg_users password hashing
State Management	Zustand / React Query	Lightweight server state caching and client state
Deployment	VPS / Docker	Cost-effective, full control, easy database connectivity
Monitoring	Sentry + Prometheus	Error tracking, performance monitoring, alerting

Table 5. Proposed Technology Stack

5. API Architecture

The REST API layer serves as the critical bridge between the Next.js frontend and the MySQL database. It must expose clean, well-documented endpoints that map directly to the existing database operations while maintaining backward compatibility with any existing API consumers using the current Laravel Sanctum tokens. The API follows RESTful conventions with consistent response formats, pagination, filtering, and error handling across all endpoints.

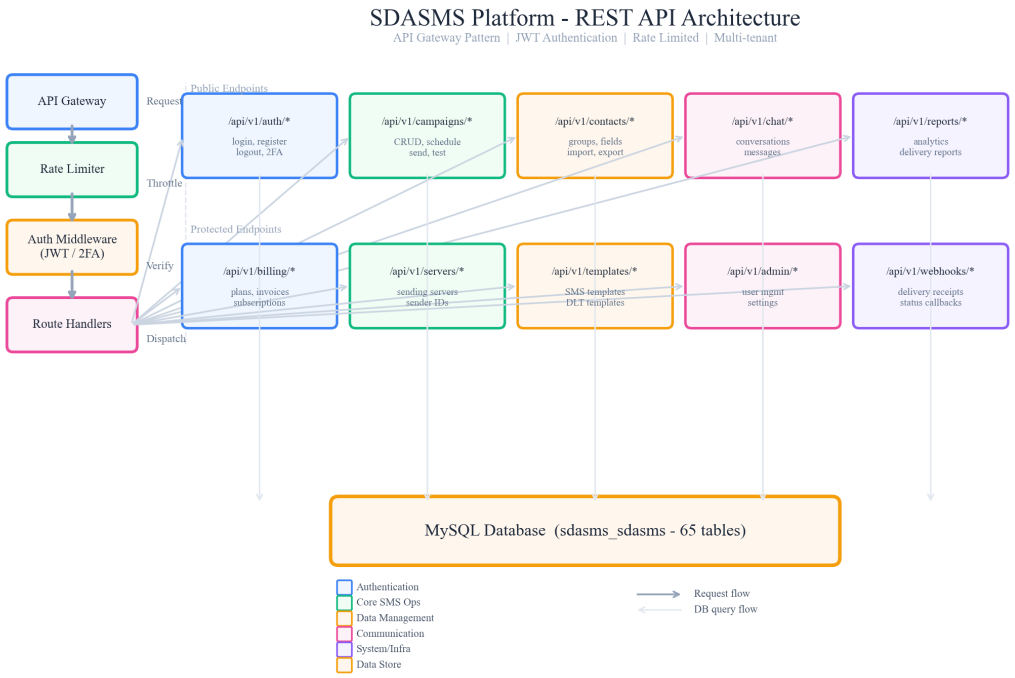

Figure 3. API Endpoint Architecture

5.1 Core API Endpoints

The following table lists the primary API endpoint groups that the backend must implement. Each group corresponds to a specific database module and provides full CRUD operations along with module-specific actions. All endpoints require authentication except where explicitly marked as public.

Endpoint Group	Key Operations	Database Tables
/api/v1/auth/*	Login, register, logout, 2FA setup, password reset, token refresh	cg_users, cg_password_resets, cg_roles
/api/v1/campaigns/*	CRUD, create/send/schedule, attach lists and sender IDs, pause/cancel	cg_campaigns, cg_campaigns_lists, cg_campaigns_senderids, cg_file_campaign_data
/api/v1/contacts/*	CRUD groups and contacts, custom fields, import CSV, opt-in/out keywords	cg_contacts, cg_contact_groups, cg_contact_group_fields, cg_contacts_custom_field
/api/v1/chat/*	List conversations, get/send messages, pin conversations, real-time via WS	cg_chat_boxes, cg_chat_box_messages
/api/v1/reports/*	Delivery reports, campaign analytics, tracking logs, export CSV	cg_reports, cg_tracking_logs
/api/v1/billing/*	List plans, subscribe, view invoices, transaction history, payment	cg_plans, cg_subscriptions, cg_invoices, cg_payment_methods
/api/v1/servers/*	CRUD sending servers, test connection, configure custom params	cg_sending_servers, cg_custom_sending_servers

/api/v1/templates/*	CRUD SMS templates, DLT template management, tag variables	cg_templates, cg_template_tags
/api/v1/senderids/*	CRUD sender IDs, submit for approval, billing status	cg_senderid, cg_senderid_plans
/api/v1/admin/*	User management, system settings, announcements, roles and permissions	cg_users, cg_app_config, cg_roles, cg_permissions
/api/v1/webhooks/*	Delivery receipt handler, inbound SMS handler, webhook configuration	cg_reports, cg_chat_boxes

Table 6. Core API Endpoint Groups

6. Frontend Module Design

The Next.js frontend will be organized into modular pages that map directly to the database modules identified in the analysis. Each module consists of a main listing page, detail/edit pages, and supporting components. The application will use a sidebar navigation layout with module-level access control based on the user's role and permissions stored in the cg_roles and cg_permissions tables. The following table outlines the planned page structure for each module, ensuring comprehensive coverage of all platform functionality.

Module	Pages	Key Features
Dashboard	home / dashboard	SMS usage stats, credit balance, recent campaigns, delivery rates, quick actions
Campaigns	campaigns	Create, schedule, send campaigns; attach contact lists; select sender IDs; DLT template picker
Contacts	contacts	Contact groups, custom fields, CSV import, opt-in/opt-out keywords, segment builder
Chat	chat	Two-way SMS conversations, real-time messaging, pinned chats, media support
Reports	reports	Delivery analytics, campaign performance, export to CSV/PDF, date range filters
Templates	templates	SMS template CRUD, variable tags, DLT template linking, approval status
Sender IDs	sender-ids	Apply for sender IDs, track approval, billing info, entity documents
Billing	billing	View plans, subscribe, pay invoices, view transaction history
API Keys	api-keys	Generate/regenerate API tokens, view usage logs, set webhook URL
Settings	settings	Profile, 2FA, timezone, language, notification preferences
Admin Panel	admin	User management, system config, sending servers, payment methods, announcements

Table 7. Frontend Page Structure by Module

7. Database Access Strategy

Choosing the right database access strategy is one of the most critical decisions for this project, as it directly impacts security, performance, and the ability to run both platforms simultaneously during migration. Three approaches were evaluated, with the API middleware approach recommended as the optimal solution.

7.1 Recommended: API Middleware Layer

The recommended approach places a Node.js API layer between the Next.js frontend and the MySQL database. The API layer handles all database operations including CRUD operations, complex queries with joins, transaction management, and data validation. This provides several critical advantages: the database credentials are never exposed to the frontend, rate limiting can be applied at the API level to prevent abuse, caching can be implemented for frequently accessed data like user settings and contact lists, and the API can maintain backward compatibility with any existing Laravel Sanctum tokens that external integrations may be using. The API layer can also implement database connection pooling, query optimization, and read replicas for reporting-heavy operations without changing the frontend code.

Criterion	Direct Connection	API Middleware (Recommended)
Security	DB credentials exposed	Credentials isolated in backend
Performance	Low latency, direct	Slight overhead, cacheable
Compatibility	Risk of schema conflicts	Controlled data access layer
Rate Limiting	Not possible	Built-in at API level
Concurrent Writes	High risk of conflicts	Managed through transactions
Migration Path	Difficult to change later	Easy to evolve independently
Old Platform Support	Both write directly	Controlled coexistence

Table 8. Database Access Strategy Comparison

8. Migration Strategy

The migration from the existing Laravel platform to the new Next.js application must be executed carefully to ensure zero data loss and minimal service disruption. The strategy follows a phased

approach where both platforms run simultaneously against the same database, with users gradually migrated to the new interface. The existing Laravel application continues to function normally throughout the transition, ensuring business continuity.

8.1 Phase 1: Foundation (Weeks 1-3)

The first phase focuses on establishing the technical foundation. This includes setting up the Node.js backend with Prisma ORM connected to the existing MySQL database, implementing authentication that reads from and validates against the cg_users table (using bcrypt password verification compatible with Laravel's hashing), building the basic Next.js application shell with sidebar navigation and layout components, and deploying the staging environment. The critical deliverable of this phase is a working authentication flow that can log users into the new platform using their existing credentials, proving that database connectivity and compatibility are established.

8.2 Phase 2: Core Features (Weeks 4-8)

The second phase builds the core business modules: campaigns, contacts, and reporting. This includes the campaign composer with contact list selection, sender ID assignment, message template usage, and scheduling capabilities. The contact management module must support CSV import (matching the existing cg_csv_data pattern), custom field management, and group-based opt-in/opt-out keywords. The reporting module should provide delivery analytics with charts and export capabilities, reading from the cg_reports and cg_tracking_logs tables. The two-way chat module with WebSocket integration should also be implemented in this phase.

8.3 Phase 3: Billing and Admin (Weeks 9-11)

The third phase implements the billing module (plans, subscriptions, invoices, payment processing), the admin panel (user management, system settings, sending server configuration), and remaining features like sender ID management, keyword handling, phone number provisioning, and the automation engine. The admin panel must provide full control over all platform settings stored in cg_app_config, user role management through cg_roles and cg_permissions, and sending server configuration through cg_sending_servers and cg_custom_sending_servers.

8.4 Phase 4: Testing and Launch (Weeks 12-14)

The final phase focuses on comprehensive testing, performance optimization, and production deployment. This includes end-to-end testing of all workflows (campaign creation to delivery report verification), load testing for bulk SMS sending operations, security auditing of the API layer, and setting up monitoring and alerting. User migration is executed gradually, starting with admin users, then power users, and finally all remaining users. The existing Laravel platform remains available as a fallback throughout the launch period, with a final cutover planned after 30 days of stable operation of the new platform.

9. Security Considerations

Running two applications against the same database introduces specific security challenges that must be addressed proactively. The following measures are essential for maintaining data integrity and preventing unauthorized access.

Concern	Risk	Mitigation
Concurrent Writes	Data corruption when both platforms modify same records	Database transactions with row-level locking, optimistic concurrency control
Authentication	Password hash compatibility between Laravel and Node.js	Use bcrypt with matching cost factor (Laravel default: 10 rounds)
API Tokens	Existing Sanctum tokens must work with new API	Validate tokens against cg_personal_access_tokens table
SQL Injection	Direct database access increases attack surface	Use Prisma parameterized queries exclusively, never raw SQL
Rate Limiting	Abuse of SMS sending endpoints	Per-user rate limits on campaign creation and API sending endpoints
Data Privacy	Contact numbers and messages contain PII	Encryption at rest, field-level encryption for phone numbers
Schema Changes	New platform modifying schema breaks old platform	Read-only constraint on new platform; all schema changes via migration scripts
Webhook Security	Fake delivery receipts from unauthorized sources	HMAC signature verification on all webhook endpoints

Table 9. Security Considerations and Mitigations

10. Development Roadmap

The following roadmap provides a high-level timeline for the complete platform rebuild, organized into four phases over approximately 14 weeks. Each phase has clear deliverables and dependencies on the previous phase, ensuring a structured and predictable development process.

Phase	Duration	Key Deliverables	Dependencies
Phase 1: Foundation	Weeks 1-3	Auth, DB connection, app shell, staging deploy	MySQL remote access enabled
Phase 2: Core Features	Weeks 4-8	Campaigns, contacts, chat, reporting modules	Phase 1 auth working
Phase 3: Billing and Admin	Weeks 9-11	Subscriptions, invoices, admin panel, automations	Phase 2 core features

Phase 4: Testing and Launch	Weeks 12-14	E2E testing, load testing, security audit, production deploy	All phases complete

Table 10. Development Roadmap Summary

This architecture document provides the comprehensive blueprint needed to build the new Bulk SMS platform. The existing database of 65 tables is well-structured and follows consistent Laravel conventions, making it highly suitable for integration with a modern Next.js and Node.js application stack. By following the API middleware approach and the phased migration strategy outlined in this document, the transition can be executed with minimal risk and zero downtime for existing users.